

FlameGuy Combat Report

This report combines your gameplay observations from *Prince of Persia: The Lost Crown* and *Dead Cells* with design conclusions for the current Unity project, with a specific focus on FlameGuy.

Project: 2DGame

Date: 2026-05-07

Executive Summary

The strongest shared takeaway from both reference games is that enemies do not behave like raw physics objects. They behave like combat actors. Positioning, spacing, pushback, cooldowns, and retreat behavior are deliberately authored. Normal contact is tightly controlled, while combat displacement is intentional and readable.

Core conclusion: FlameGuy should be designed as a **combat actor**, not as a regular dynamic rigidbody enemy that passively accepts player push forces.

Observed Reference Behaviors

Prince of Persia: The Lost Crown

- Enemies do not visibly slide around from simple body contact.
- Some enemies create space after attacks. Sometimes the player is displaced, sometimes the enemy steps back.
- Enemy attack archetypes vary: charge attacks, distance-closing runs, dashes, and standard strikes.
- When the player runs directly into an enemy, the enemy often gives ground slowly to preserve spacing.
- There are clear attack cooldowns, often around one to two seconds between actions.
- Some enemies have combo attack chains.
- Charge attacks are designed to be answered with systems like parry or dash.
- Player melee attacks include forward lunges and controlled pushback that preserve combat readability.

Dead Cells

- Player and enemy collision is not treated like persistent body-blocking.
- During enemy attack phases, collision feels even more relaxed or disabled.
- Enemies generally do not fall off platforms under normal combat pressure.

- Knockback exists as a combat reaction, but platform-fall knockback effectively acts like a lethal outcome.
- Attacks still include forward lunges.
- Combo chains are shorter, but the timing between attacks is very deliberate.

Shared Design Pattern

Although the two games feel different, they share the same structural logic:

1. Normal body push is weak or absent.
2. Combat displacement is strong, but only when intentionally triggered.
3. Enemy spacing is authored, not left to the physics solver.
4. Platform ownership is stable. Enemies do not casually drift off ledges.
5. Cooldowns and recovery windows shape the combat rhythm.
6. Enemy movement is state-driven rather than force-driven.

Implications for This Project

The current Unity setup has several archetypes that were built at different times and therefore follow different combat rules. Your observations now give us a clearer standard to converge toward.

- Enemies should not be freely pushed by the player during normal contact.
- If spacing changes, that should happen because of an attack, recoil, retreat, dash, or a dedicated reposition rule.
- Platform loss should be a special-case behavior, not an accidental byproduct of physics.
- Basic melee pressure should preserve readable distance instead of collapsing characters into each other.

FlameGuy Role Assessment

FlameGuy is a low-mobility zone-control enemy. That makes it a strong candidate for a more authored movement model than a pure dynamic rigidbody combatant. Its identity benefits from holding position, owning space, and punishing the player with a flame lane, rather than being shoved around during close contact.

- FlameGuy should feel grounded and stubborn.
- FlameGuy should keep pressure through attack commitment.
- FlameGuy should not be casually displaced by the player's body.
- FlameGuy should preserve the platform or lane it was designed to control.

Recommended Technical Direction for FlameGuy

The long-term direction should be to treat FlameGuy as a rule-driven combat actor with explicit spacing and platform ownership.

1. Movement Model

- Prefer an authored movement model over passive force-based reactions.
- Normal position updates should come from AI state logic, not from player push forces.
- If a kinematic approach is adopted later, movement should be driven through controlled patrol and reposition rules.

2. Platform Ownership

- FlameGuy should not casually fall from its patrol lane.
- `leftPoint` and `rightPoint` already provide a strong base for lane ownership.`
- If needed, an additional ground-ahead check can stop it from moving off ledges.

3. Combat Spacing

- Add a preferred combat distance.
- Add a minimum distance threshold for pressure response.
- If the player enters too deeply, FlameGuy should use a short retreat or spacing correction rather than raw physics drift.

4. Attack Loop

- Patrol
- Prep
- Flame
- Recovery / Cooldown
- Optional short retreat before re-entering patrol or idle combat watch

5. Hit Reaction Policy

- FlameGuy should keep its attack commitment when hit, according to your latest direction.
- Visual feedback such as hit flash can remain.
- If a reaction is added later, it should be small and should not invalidate the flame attack identity.

FlameGuy Behavior Spec

Patrol

- Moves between two points.
- Does not leave its intended platform.
- Transitions into prep when the player enters its cone and range conditions are met.

Prep

- Stops and faces the player.
- Maintains composure under pressure.
- Does not cancel attack commitment when hit.

Flame

- Holds its lane while executing the flame attack.
- Uses the flame area as its authored damage channel.
- Does not get body-pushed during the attack phase.

Recovery

- After the flame ends, a cooldown prevents immediate repetition.
- A short retreat may be added to restore spacing if the player is too close.

Why Mass Alone Is Not Enough

Increasing rigidbody mass can reduce push, but it does not solve the design problem. As long as the player and FlameGuy still interact through a normal dynamic rigidbody contact model, the player can continue to influence it in ways that do not match the reference games.

- Mass tuning may reduce displacement but will not fully eliminate it.
- Mass tuning can make future knockback balancing harder.
- Mass does not introduce authored spacing or retreat behavior.

Current Best Direction

Based on your reference notes, the project should move away from “heavy physics enemy” thinking and toward “authored combat actor” thinking. FlameGuy is the best first archetype for that shift because it naturally benefits from stable platform ownership and intentional spacing control.

Suggested Next Steps

1. Define a FlameGuy spacing policy: preferred distance, minimum distance, retreat duration, and cooldown duration.
2. Decide whether FlameGuy should become fully kinematic or keep a hybrid movement model with platform ownership rules.
3. Implement a post-attack recovery window.
4. Implement pressure response that preserves spacing without raw body push.
5. Apply the resulting combat standard later to MeleeEnemy and other grounded archetypes.

Prepared from your gameplay observations and translated into project-facing combat design recommendations for the current Unity codebase.